Second, *discrepancy* [93] gives the maximum disagreement between two classifiers from a set of multiple models:

$$m_{\text{Discrepancy}}(\mathcal{O}) = \max_{\substack{\boldsymbol{O},\boldsymbol{O'}\in\mathcal{O} \\ s.t.\ \boldsymbol{O}\neq\boldsymbol{O'}}} \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\{\arg\max_j \boldsymbol{O}_{i,j} \neq \arg\max_j \boldsymbol{O'}_{i,j}\}. \tag{56}$$

Both ambiguity and discrepancy are bounded in the interval $[0, 1]$, with a value of zero indicating perfect agreement.

Label Entropy. Datta et al. [31] measured the variance in individual predictions over a group of outputs in terms of entropy. Letting $k_c^{(i)}$ denote the number of times that instance i is predicted as class c, *Label Entropy* (LE) is defined as

$$m_{\text{LE}}(\mathcal{O}, i) = \sum_{c=1}^{C} -\frac{k_c^{(i)}}{|\mathcal{O}|} \log\left(\frac{k_c^{(i)}}{|\mathcal{O}|}\right). \tag{57}$$

Label Entropy is bounded in the interval $[0, \log(C)]$, with $m_{\text{LE}}(\mathcal{O}, i) = 0$ indicating identical predictions.

4.3 Soft Prediction-Based Measures

This group of measures compares soft predictions, such as class-wise probabilities or scores from decision functions. Intuitively, this provides more nuance to the notion of similarity in outputs, since we can consider differences in confidence of individual predictions. The impact of confidence is specifically exemplified by cases where scores are close to the decision boundary. Even a minimal change in scores may cause a different classification in one case, whereas scores would need to change drastically for a different classification in another case.

Norm of Soft Prediction Difference. A direct way to generalize disagreement to soft predictions is to apply a norm $\|\cdot\|$ on instance-wise differences in soft predictions, and average this over all inputs, which yields a measure

$$m_{\text{PredNormDiff}}(\boldsymbol{O}, \boldsymbol{O'}) = \frac{1}{2N}\sum_{i=1}^{N} \|\boldsymbol{O}_i - \boldsymbol{O'}_i\| \tag{58}$$

that assigns a score of zero when outputs are equal. Ba and Caruana [4] and Zhang et al. [163] applied this measure using the Euclidean norm to compare logits and probabilities, respectively.

Surrogate Churn. Bhojanapalli et al. [14] proposed *surrogate churn* (SChurn) as a relaxed version of disagreement, that takes into account the distribution of the soft predictions. For $\alpha > 0$, it is defined as

$$m^{\alpha}_{\text{SChurn}}(\boldsymbol{O}, \boldsymbol{O'}) = \frac{1}{2N}\sum_{i=1}^{N} \left\| \left(\frac{\boldsymbol{O}_i}{\max_c \boldsymbol{O}_{i,c}}\right)^{\alpha} - \left(\frac{\boldsymbol{O'}_i}{\max_c \boldsymbol{O'}_{i,c}}\right)^{\alpha} \right\|_1. \tag{59}$$

A value $m^{\alpha}_{\text{SChurn}}(\boldsymbol{O}, \boldsymbol{O'}) = 0$ indicates perfect agreement of outputs. The authors showed that when $\alpha \to \infty$, this measure is equivalent to standard disagreement (cf. Sec. 4.2), and use $\alpha = 1$ as the default value.

Divergence-Based Measures. When soft predictions represent class probabilities, divergence measures for probability distributions can be used to evaluate the similarity of instance-level predictions. For example, Kullback-Leibler divergence is commonly used when training similar models in knowledge distillation [49]. When focusing on pure similarity assessment, a common choice is to apply the symmetric *Jensen-Shannon Divergence* (JSD) by averaging over all instances [36, 43, 156]. Letting $\text{KL}(\cdot\|\cdot)$ denote the Kullback-Leibler divergence, this measure is defined as

$$m_{\text{JSD}}(\boldsymbol{O}, \boldsymbol{O'}) = \frac{1}{2N}\sum_{i=1}^{N} \text{KL}(\boldsymbol{O}_i \| \overline{\boldsymbol{O}}_i) + \text{KL}(\boldsymbol{O'}_i \| \overline{\boldsymbol{O}}_i), \tag{60}$$

with $\overline{\boldsymbol{O}} = \frac{\boldsymbol{O}+\boldsymbol{O'}}{2}$ denoting the average output. Equality of outputs is given when $m_{\text{JSD}}(\boldsymbol{O}, \boldsymbol{O'}) = 0$, and higher values indicate dissimilarity. A similar approach to compare probabilistic outputs is given as *Graph Explanation Faithfulness* (GEF) [1]. An overview of divergence measures that could be used has been given by Cha [23].

Prediction Difference. Shamir and Coviello [126] specifically considered differences in predictions over more than two models. Their *prediction difference* (PD) intuitively quantifies the variance in model predictions. Letting $\overline{\boldsymbol{O}} = \frac{1}{|\mathcal{O}|}\sum_{\boldsymbol{O}\in\mathcal{O}} \boldsymbol{O}$ denote the average output matrix, their standard prediction difference measure aggregates instance-wise deviations from the average output in terms of a p-norm:

$$m^{p}_{\text{PD}}(\mathcal{O}) = \frac{1}{N}\sum_{i=1}^{N} \frac{1}{|\mathcal{O}|}\sum_{\boldsymbol{O}\in\mathcal{O}} \|\boldsymbol{O}_i - \overline{\boldsymbol{O}}_i\|_p. \tag{61}$$

Shamir and Coviello [126] used $p = 1$ for interpretable differences of probability distributions. $m^{p}_{\text{PD}}(\mathcal{O}) = 0$ indicates identical outputs of all models. Higher PD indicates higher dissimilarity between the compared models.

Next to norm-based prediction difference, Shamir and Coviello [126] further proposed a variant of the PD that relates the variance in the outputs to their average magnitude, and a variant that considers class labels $\boldsymbol{y}$ if these are given.

Rashomon Capacity. Similar to label entropy (Eq. 57), *Rashomon Capacity* (RC) [62] also applies concepts from information theory to measure multiplicity in predictions on individual instances. Formally, letting $P_{\boldsymbol{O}}$ denote a

probability distribution over the set of outputs $\mathcal{O}$, and $\Delta_C = \left\{\boldsymbol{p} \in [0,1]^C : \sum_{i=1}^C \boldsymbol{p}_i = 1\right\}$ the probability simplex, it considers the output spread $\inf_{\boldsymbol{p}\in\Delta_C} \mathbb{E}_{\boldsymbol{O}\sim P_{\boldsymbol{O}}} \mathrm{KL}(\boldsymbol{O}_i \| \boldsymbol{p})$, where $\boldsymbol{p} \in \Delta_C$ is a reference distribution that is optimized to minimize distances to all outputs. The Rashomon Capacity is then defined via the *channel capacity*, which maximizes the output spread over all probability distributions over the outputs:

$$m_{\text{RC}}(\mathcal{O}, i) = 2^{\text{Capacity}(\mathcal{O},i)}, \quad \text{with} \quad \text{Capacity}(\mathcal{O}, i) = \sup_{P_{\boldsymbol{O}}} \inf_{\boldsymbol{p}\in\Delta_C} \mathbb{E}_{\boldsymbol{O}\sim P_{\boldsymbol{O}}} \mathrm{KL}(\boldsymbol{O}_i \| \boldsymbol{p}). \tag{62}$$

To approximate the Rashomon Capacity of an instance, Hsu and Calmon [62] suggested using the Blahut–Arimoto algorithm [3, 16]. A similarity measure over all instances can be obtained by aggregation, e.g., via the mean value.

It holds that $m_{\text{RC}}(\mathcal{O}, i) \in [1, C]$ with $m_{\text{RC}}(\mathcal{O}, i) = 1$ if and only if all outputs are identical, and $m_{\text{RC}}(\mathcal{O}, i) = C$ if and only if every class is predicted once with perfect confidence. Further, the measure is monotonous, i.e., it holds that $m_{\text{RC}}(\mathcal{O}', i) \leq m_{\text{RC}}(\mathcal{O}, i)$ for all $\mathcal{O}' \subseteq \mathcal{O}$.

4.4 Gradient and Adversarial Example-Based Measures

The measures in this section use model gradients to characterize similarity either directly or indirectly via adversarial examples. A core assumption of these measures is that similar models have similar gradients. This assumption also leads to transferability of adversarial attacks, i.e., the behavior of the compared models changes similarly when given an adversarial example computed for only one of the models.

ModelDiff. In their *ModelDiff* measure, Li et al. [82] used adversarial examples from perturbation attacks to characterize decision regions, which can then be compared across two models. Given a model f, they first created adversarial examples $\widetilde{\boldsymbol{X}}_i$ for every input $\boldsymbol{X}_i$ by adding noise to these inputs that steer the model away from a correct prediction. Such examples can be determined by methods such as projected gradient descent [91]. The difference between the instance-wise original soft predictions $\boldsymbol{O}_i = f(\boldsymbol{X}_i)$ and the predictions for the corresponding adversarial example $\widetilde{\boldsymbol{O}}_i = f(\widetilde{\boldsymbol{X}}_i)$ is then collected in a *decision distance vector* defined as $\left(\boldsymbol{v}_{\text{DDV}}(\boldsymbol{O}, \widetilde{\boldsymbol{O}})\right)_i = \text{cos-sim}(\boldsymbol{O}_i, \widetilde{\boldsymbol{O}}_i)$. Finally, they quantified the difference between models via the difference in the DDVs, measured with cosine similarity:

$$m_{\text{ModelDiff}}(\boldsymbol{O}, \boldsymbol{O'}) = \text{cos-sim}(\boldsymbol{v}_{\text{DDV}}(\boldsymbol{O}, \widetilde{\boldsymbol{O}}), \boldsymbol{v}_{\text{DDV}}(\boldsymbol{O'}, \widetilde{\boldsymbol{O}}')). \tag{63}$$

The outputs $\widetilde{\boldsymbol{O}}'_i = f'(\widetilde{\boldsymbol{X}}_i)$ are computed from the same adversarial examples $\widetilde{\boldsymbol{X}}_i$. A similarity score of one indicates equivalence of outputs. Since this measure uses adversarial examples of only one of the models, it is not symmetric.

Adversarial Transferability. Similar to ModelDiff, Hwang et al. [63] measured the similarity of networks in terms of the transferability of adversarial attacks. Given two networks f, f', for each input $\boldsymbol{X}_i$ that is predicted correctly by both networks, a pair of corresponding adversarial examples $\widetilde{\boldsymbol{X}}_i, \widetilde{\boldsymbol{X}}'_i$ is generated with projected gradient descent [91]. These adversarial examples are then fed into the opposite model, yielding outputs $\widetilde{\boldsymbol{O}}_i = f(\widetilde{\boldsymbol{X}}'_i)$ and $\tilde{\boldsymbol{O}}'_i = f'(\widetilde{\boldsymbol{X}}_i)$, for which it is then determined how often both are incorrect. Thus, given the vector of ground-truth labels $\boldsymbol{y}$, and letting $\mathcal{X}_{\text{true}}$ denote the set of instances that were predicted correctly by both models, Hwang et al. [63] defined the measure

$$m_{\text{AdvTrans}}(\widetilde{\boldsymbol{O}}, \widetilde{\boldsymbol{O}}') = \log\left[\max\left\{\varepsilon, \tfrac{100}{2|\mathcal{X}_{\text{true}}|} \textstyle\sum_{i\in\mathcal{X}_{\text{true}}} \left(\mathbb{1}(\arg\max_j \widetilde{\boldsymbol{O}}_{i,j} \neq \boldsymbol{y}_i) + \mathbb{1}(\arg\max_j \widetilde{\boldsymbol{O}}'_{i,j} \neq \boldsymbol{y}_i)\right)\right\}\right], \tag{64}$$

where $\varepsilon > 0$ is introduced to avoid $\log(0)$. A value of $m_{\text{AdvTrans}}(\widetilde{\boldsymbol{O}}, \widetilde{\boldsymbol{O}}') = \log(100)$ indicates perfect model similarity, whereas $m_{\text{AdvTrans}}(\widetilde{\boldsymbol{O}}, \widetilde{\boldsymbol{O}}') = \log(\varepsilon)$ indicates complete disagreement.

Cosine Similarity of Saliency Maps. Jones et al. [64] used a direct approach to compare models in terms of their gradients. They computed the cosine similarity between (vectorized) saliency maps [130], which model the impact of input features on individual predictions. Practically, this impact is quantified using instance-wise gradients $\nabla_{\boldsymbol{X}_i} \boldsymbol{O}_{i,c}$, and the instance-wise similarities then aggregated to yield the following measure:

$$m_{\text{SaliencyMap}}(\boldsymbol{O}, \boldsymbol{O'}) = \tfrac{1}{nC} \textstyle\sum_{i=1}^N \sum_{c=1}^C \text{cos-sim}\left(\left|\nabla_{\boldsymbol{X}_i} \boldsymbol{O}_{i,c}\right|, \left|\nabla_{\boldsymbol{X}_i} \boldsymbol{O}'_{i,c}\right|\right), \tag{65}$$

where the absolute value $|\cdot|$ is applied element-wise (for inputs with a single channel). A value $m_{\text{SaliencyMap}}(\boldsymbol{O}, \boldsymbol{O'}) = 1$ indicates perfect similarity, with lower values indicating stronger differences between models.

4.5 Stitching-Based Measures

The intuition behind *stitching* is that similar models should be similar in their internal processes and, thus, swapping layers between such models should not result in big differences in the outputs if a layer that converts representations is introduced [80, 30, 7]. Given two models f, f', stitching consists of training a *stitching layer* (or network) g to convert representations from f at layer l into representations of f' at layer l'. One then considers the composed model

$\widetilde{f} := f'^{(L')} \circ \cdots \circ f'^{(l'+1)} \circ f'^{(l')} \circ g \circ f^{(l)} \circ f^{(l-1)} \circ \cdots \circ f^{(1)}$, which uses the bottom-most layers of f and the top-most layers of f', and compares its output with the original models. Most commonly they are compared in terms of a quality function q such as accuracy [30, 7]. This yields a measure

$$m_{\text{stitch}}(\widetilde{\boldsymbol{O}}, \boldsymbol{O'}) = q(\widetilde{\boldsymbol{O}}) - q(\boldsymbol{O'}), \tag{66}$$

where $\widetilde{\boldsymbol{O}} = \widetilde{f}(\boldsymbol{X})$ is the output of the stitched model. However, other functional similarity measures can also be used.

Both design and placement of stitching layers affects assessments of model similarity, and several types of stitching layers were studied [30, 47]. Bansal et al. [7] chose stitching layers such that the architecture of the stitched model is consistent with the original models. For instance, they use a token-wise linear function to stitch transformer blocks. For CNNs, 1×1 convolutions are generally used in stitching layers [80, 30, 7].

Compared to other measures, model stitching requires training an additional layer and thus might be more costly to implement. Further, (non-deterministic) training of the stitching layer presents a source of instability of the final results. To train the stitching layers, one typically freezes parameters of the original models and only optimizes the weights of the stitching layer via gradient descent, using ground truth labels or the output of f' as soft labels [30, 7, 80]. Additional tweaks such as normalization or regularization may be beneficial in certain contexts [7, 30]. For simple linear stitching layers $\boldsymbol{T}$, the weights can be computed by solving the least squares problem $\|\boldsymbol{R}^{(l)}\boldsymbol{T} - \boldsymbol{R'}^{(l')}\|_F$.

5 Properties and Application of Similarity Measures

In this section, we discuss practical aspects regarding the application of similarity measures. We begin by outlining the current state of research that analyzes properties of existing measures and their relationship. Afterwards, we summarize applications in existing literature, and then discuss the choice of measures in more detail, before providing additional considerations for comparing neural networks.

Properties and Evaluation of Similarity Measures. Most research on properties of similarity measures in the deep learning literature focuses on representational similarity. We give a detailed review of existing analyses of representational similarity measures in Appendix C. We further provide an overview of existing comparative tests of these measures in Table 3, which highlights that except for the recent `ReSi` benchmark [70], most analyses only considered very limited sets of measures. By contrast, functional similarity measures have been broadly analyzed in various contexts, including inter-rater agreement [129, 135, 89], model fingerprinting [138], and ensemble learning [76].

Resources. There are only few resources that enable easy use of similarity measures. Most notably, the recent `ReSi` benchmark[1] [70] provides implementations of 24 representational similarity measures, and also allows for testing of new measures on representations from a broad range of neural network models and datasets, spanning the graph, language, and vision domains. Similarly, Ding et al. [34] provide code[2] to replicate their experiments and test new measures, albeit being smaller in scope. Finally, Cloos et al. [26] have collected implementations of similarity measures in an online repository[3], aiming to provide a standardized interface for application of existing measures.

Applications in the Literature. Similarity measures have been used in a wide array of contexts with two main objectives: to *understand* aspects of deep learning, and to *improve* deep learning systems. In this section, we give an overview and examples of such applications; for a wider overview we refer to Sucholutsky et al. [136, Section 4].

Focal points of work aiming at *understanding* deep learning include the effects of model architecture and objective function on what neural networks learn, as well as studies on model universality, i.e., the extent models converge to similar behavior under different training setups. For example, Raghu et al. [115] and Park and Kim [111] studied the differences between vision transformers and CNNs by comparing their representations and analyzing changes in classification performance after modifying the architectures. Similarly, the effect of width and depth [107] and the importance of specific layers [133] has been investigated. The impact of differences in the objective functions has, for instance, been studied by Kornblith et al. [72], who analyzed layer-wise differences in representations between models that vary only in their loss function, and further evaluated transferability of these models via functional similarity measures. Grigg et al. [51] took a similar approach when comparing supervised to self-supervised models. Some studies have also investigated the impact of training adversarially robust models, by considering how intra- [25] and inter-architecture [64] similarities of representations from robust and non-robust models differ, or how stitching these kinds of models affects performance [5]. Finally, studies on model universality have found that neural networks trained under different training setups are often at least partially similar in their representations [156, 97, 99, 137, 98], even if

[1] `https://github.com/mklabunde/resi`

[2] `https://github.com/js-d/sim_metric`

[3] `https://github.com/nacloos/similarity-repository`